



PROGRAMAÇÃO



Escola Naval



António Gonçalves

Agenda



Listas

Criação, índices
e alteração



Processamento

Pesquisa, métodos
e compreensão



Organização

Funções, ficheiros
e listas de listas



Tuplos e gráficos

Imutabilidade, conversão
e Matplotlib



Exercício final com notas de alunos.

Uma lista reúne vários valores

Uma **lista** é uma sequência ordenada e mutável de valores. Os elementos são identificados por um índice, começando em zero.

		lista de notas				
		0	1	2	3	4
índices	valores	14	16	12	18	15
		notas				

Criação de uma lista

```
notas = [14, 16, 12, 18, 15]
```

Outros exemplos de sintaxe

```
vazia = [] # lista vazia  
nomes = ['Ana', 'Rui'] # lista de cadeias  
dados = ['Ana', 18, True] # tipos diferentes
```

As listas podem conter valores de tipos diferentes.

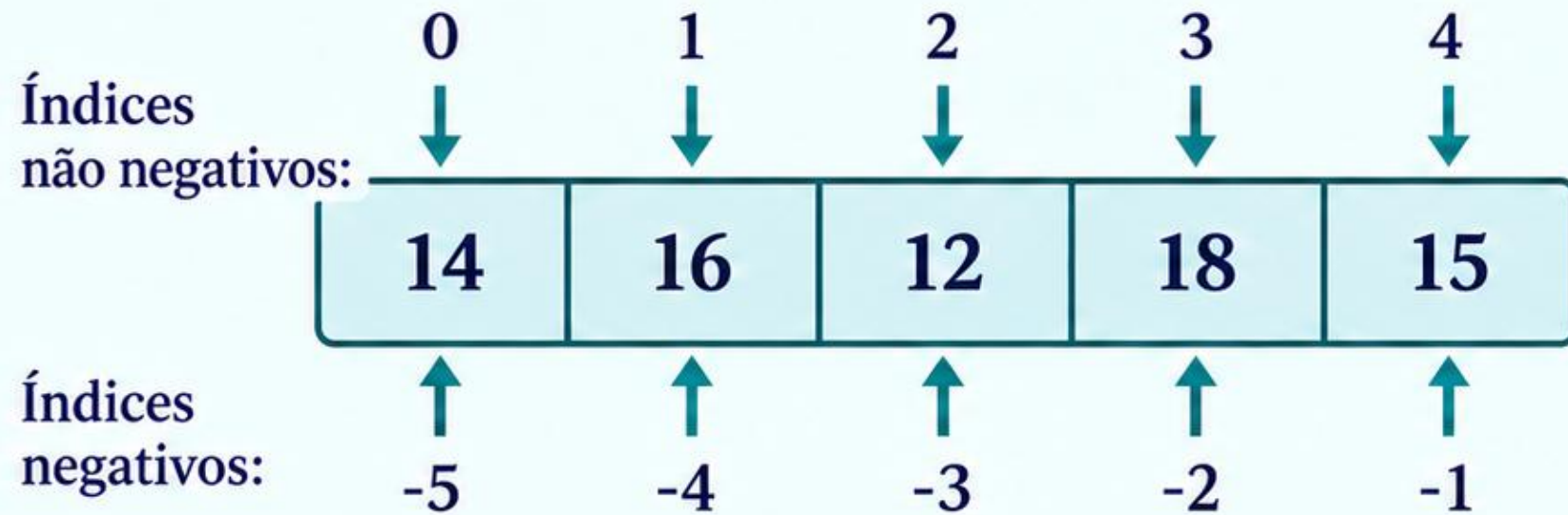
Expressões de criação de listas

Expressão	Resultado
<code>list(range(1, 6))</code>	<code>[1, 2, 3, 4, 5]</code>
<code>[0] * 3</code>	<code>[0, 0, 0]</code>
<code>[1, 2] + [3, 4]</code>	<code>[1, 2, 3, 4]</code>

Uma **lista** mantém a ordem dos elementos e permite alterar o seu conteúdo após a criação.

ÍNDICES E ACESSO AOS ELEMENTOS

Lista: `notas = [14, 16, 12, 18, 15]`



Aceder a um índice fora do intervalo provoca um erro:

`notas[5]`



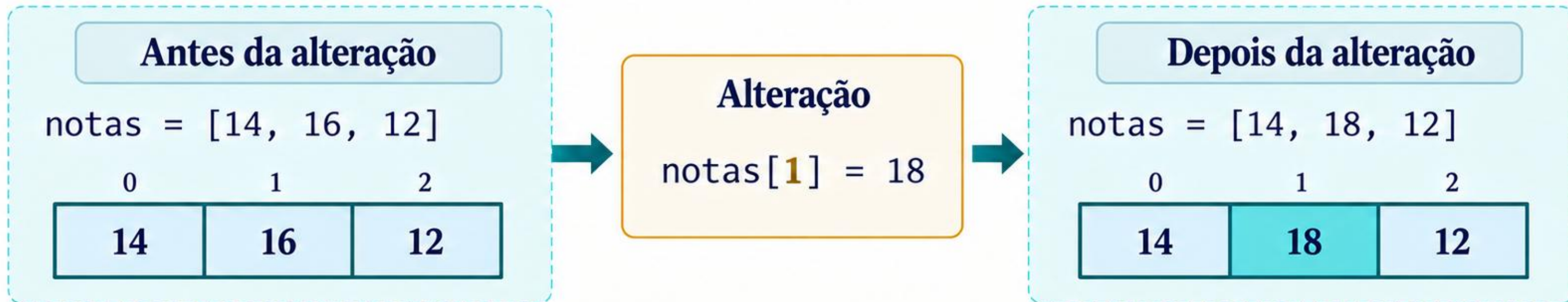
IndexError

Exemplos de acesso:

Expressão (Python)	Resultado
<code>notas[0]</code>	14
<code>notas[-1]</code>	15
<code>len(notas)</code>	5
<code>notas[len(notas) - 1]</code>	15

Índices não negativos válidos:
de **0** a **`len(notas) - 1`**.

ALTERAR E PERCORRER UMA LISTA



Por valor

```
for nota in notas:  
    print(nota)
```

Resultado

14
18
12

Por índice

```
for i in range(len(notas)):  
    print(i, notas[i])
```

Resultado

0 14
1 18
2 12

Por valor quando basta ler; **por índice** quando é preciso conhecer a posição.

LER E VALIDAR VALORES

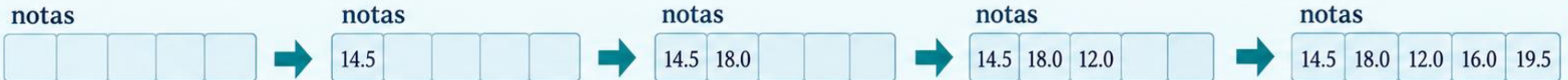
```
notas = []  
while len(notas) < 5:  
    try:  
        nota = float(input('Nota: '))  
        if 0 <= nota <= 20:  
            notas.append(nota)  
        else:  
            print('Use valores de 0 a 20.')  
    except ValueError:  
        print('Introduza um número.')
```

1 A função **input()** devolve um texto introduzido pelo utilizador.

2 A função **float()** converte o texto num número (do tipo real).

3 O método **append()** acrescenta apenas as notas válidas à lista.

Preenchimento da lista de notas

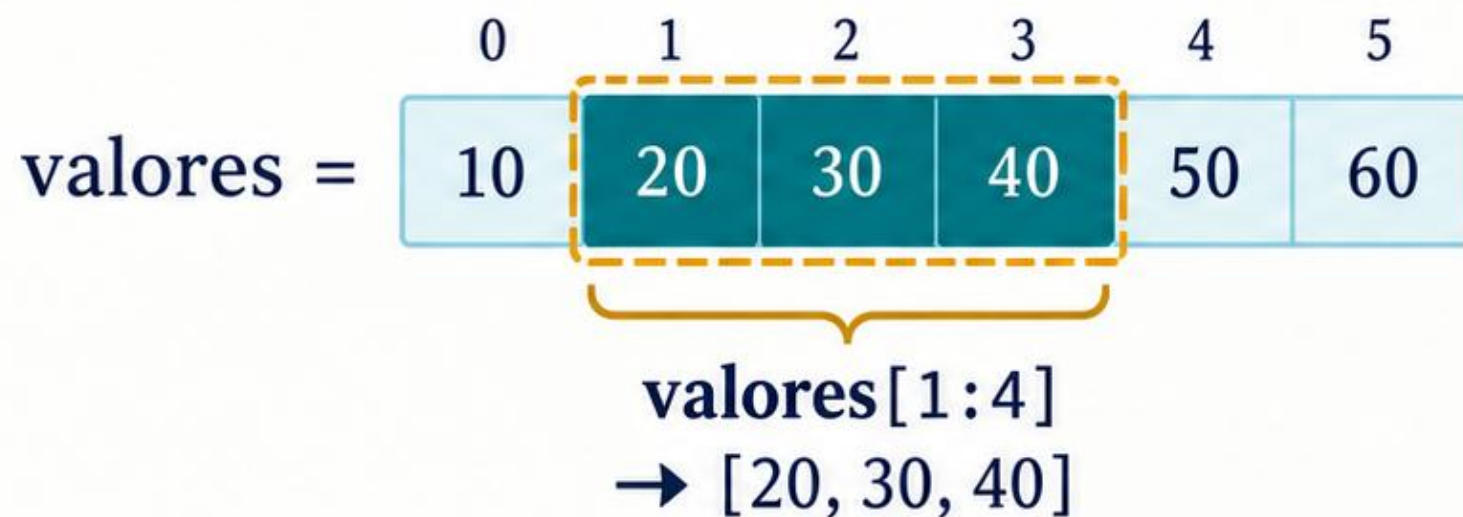


Termina quando a lista contém **5** notas válidas.

FATIAS DE UMA LISTA

Sintaxe: `lista[início:fim:passo]`

- **início:** índice do primeiro elemento incluído na fatia.
- **fim:** índice do primeiro elemento não incluído na fatia.
- **passo:** distância entre elementos (por omissão é 1).



`valores = [10, 20, 30, 40, 50, 60]`

Expressão	Resultado
<code>valores[1:4]</code>	[20, 30, 40]
<code>valores[:3]</code>	[10, 20, 30]
<code>valores[3:]</code>	[40, 50, 60]
<code>valores[::2]</code>	[10, 30, 50]
<code>valores[-3:]</code>	[40, 50, 60]
<code>valores[::-1]</code>	[60, 50, 40, 30, 20, 10]

Uma fatia cria uma **nova lista**.

Pesquisar e contar ocorrências

Lista de nomes: uma coleção ordenada de valores de texto.

```
nomes = ['Ana', 'Rui', 'Ana']
```

Podemos verificar se um valor existe na lista, obter a sua primeira posição e contar quantas vezes ocorre.

Expressão	Resultado	Significado
'Ana' in nomes	True	testa presença
'Eva' not in nomes	True	testa ausência
nomes.index('Ana')	0	primeira posição
nomes.count('Ana')	2	número de ocorrências

Código

```
if 'Rui' in nomes:  
    print(nomes.index('Rui'))
```

Resultado

1



index provoca **ValueError** se o valor não existir.

Acrescentar e remover elementos

Listas:

podem ser alteradas, permitindo acrescentar e remover elementos durante a execução do programa.

Lista inicial em cada exemplo:

```
valores = [10, 20, 30]
```



valores

Operação	Efeito	Resultado
<code>valores.append(40)</code>	acrescenta no fim da lista.	<code>[10, 20, 30, 40]</code>
<code>valores.insert(1, 15)</code>	insere na posição 1.	<code>[10, 15, 20, 30]</code>
<code>valores.remove(20)</code>	remove a primeira ocorrência do valor.	<code>[10, 30]</code>
<code>del valores[0]</code>	remove por índice.	<code>[20, 30]</code>



- Cada exemplo parte da lista inicial.
- O método `remove` usa um valor; `del` usa uma posição.
- Remover um valor inexistente causa `ValueError`.

ORDENAR E INVERTER

Vamos considerar, em cada exemplo, uma lista inicial com: `valores = [30, 10, 20]`

Operação	Código Python	Resultado	Efeito
Ordenar por ordem crescente	<code>valores = [30, 10, 20]</code> <code>valores.sort()</code>	<code>valores</code> fica <code>[10, 20, 30]</code>	Altera a lista original.
Inverter a ordem atual	<code>valores = [30, 10, 20]</code> <code>valores.reverse()</code>	<code>valores</code> fica <code>[20, 10, 30]</code>	Inverte a ordem atual da lista.
Obter uma nova lista ordenada	<code>valores = [30, 10, 20]</code> <code>nova = sorted(valores)</code>	<code>nova</code> fica <code>[10, 20, 30]</code> e <code>valores</code> fica <code>[30, 10, 20]</code>	Cria uma nova lista, sem alterar a lista original.



Atenção! Os métodos `sort()` e `reverse()` devolvem `None`.

Não escrever: `valores = valores.sort()`



Para ordenar por ordem decrescente:

```
valores = [30, 10, 20]
valores.sort(reverse=True)
```


ATRIBUIÇÃO E CÓPIA

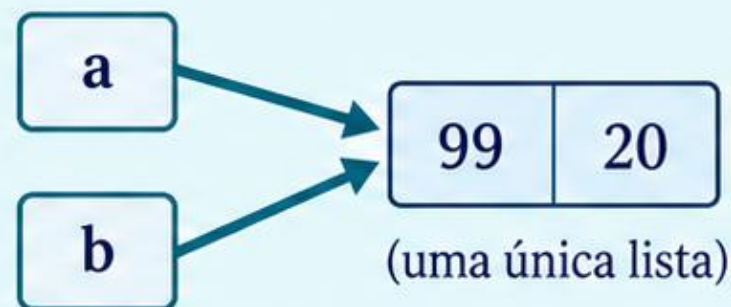
1 Dois nomes, a mesma lista

A variável **b** passa a referir o mesmo objeto que a variável **a**.

Código:

```
a = [10, 20]
b = a
b[0] = 99
```

Memória



Resultado:

```
a → [99, 20]
b → [99, 20]
```

Ambas as variáveis referem a mesma lista.

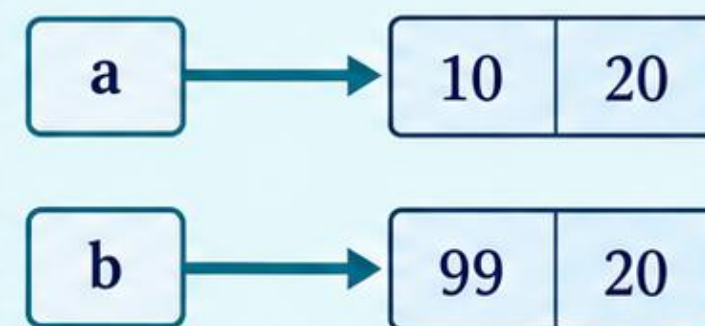
2 Uma nova lista

A variável **b** passa a referir uma cópia da lista, distinta da lista original.

Código:

```
a = [10, 20]
b = a[:]
b[0] = 99
```

Memória



Resultado:

```
a → [10, 20]
b → [99, 20]
```

Cada variável refere uma lista diferente.

a[:] e **a.copy()** fazem cópias superficiais: listas interiores continuam partilhadas.

Calcular estatísticas

Lista de notas:

```
notas = [8, 12, 15, 7, 18]
```



Resultados:

len(notas)

5

sum(notas)

60

min(notas)

7

max(notas)

18

Calcular a média com verificação:

```
if notas:  
    media = sum(notas) / len(notas)  
    print(media)  
else:  
    print('Sem notas.')
```



Saída:

12.0

Soma acumulada (alternativa):

```
total = 0  
for nota in notas:  
    total += nota
```

Traço da execução:

Iteração	nota	total (antes)	total (depois)
1	8	0	8
2	12	8	20
3	15	20	35
4	7	35	42
5	18	42	60

Atenção: A lista vazia exige cuidado: não há média, mínimo ou máximo.

CONTAR E SELECIONAR VALORES

`notas = [8, 12, 15, 7, 18]`

8	12	15	7	18
rejeitado	selecionado	selecionado	rejeitado	selecionado

1 Contar quantas notas são maiores ou iguais a 10.

```
quantidade = 0
for nota in notas:
    if nota >= 10:
        quantidade += 1
```

Iteração	nota	nota >= 10	quantidade
1	8	Falso	0
2	12	Verdadeiro	1
3	15	Verdadeiro	2
4	7	Falso	2
5	18	Verdadeiro	3

Resultado: quantidade = 3

2 Selecionar as notas maiores ou iguais a 10, construindo uma nova lista.

```
aprovados = []
for nota in notas:
    if nota >= 10:
        aprovados.append(nota)
```

Iteração	nota	nota >= 10	aprovados
1	8	Falso	[]
2	12	Verdadeiro	[12]
3	15	Verdadeiro	[12, 15]
4	7	Falso	[12, 15]
5	18	Verdadeiro	[12, 15, 18]

Resultado: aprovados = [12, 15, 18]

A mesma condição pode produzir uma contagem ou uma nova lista.

LISTAS EM FUNÇÕES

```
def selecionar_pares(valores):  
    pares = []  
    for valor in valores:  
        if valor % 2 == 0:  
            pares.append(valor)  
    return pares
```

```
numeros = [1, 2, 3, 4]  
resultado = selecionar_pares(numeros)
```



A função devolve uma nova lista e mantém **numeros**.

Execução da função

Iteração	valor	valor % 2 == 0	pares (após iteração)
–	–	–	[]
1	1	False	[]
2	2	True	[2]
3	3	False	[2]
4	4	True	[2, 4]



Atenção:

Uma função que usa **append** na lista recebida pode alterar a lista do chamador.

COMPREENSÃO DE LISTAS

Sintaxe principal: [expressão **for** elemento **in** sequência **if** condição]

1 Exemplo 1: gerar uma lista de quadrados.

```
quadrados = [n ** 2 for n in range(1, 5)]
```



Resultado: [1, 4, 9, 16]

2 Exemplo 2: selecionar os números pares.

```
pares = [n for n in range(1, 7) if n % 2 == 0]
```



Resultado: [2, 4, 6]

Equivalente com ciclo explícito (exemplo 2):

```
pares = []
for n in range(1, 7):
    if n % 2 == 0:
        pares.append(n)
```

Execução passo a passo (exemplo 2):

n	n % 2 == 0	Ação	pares
1	False	não adiciona	[]
2	True	adiciona 2	[2]
3	False	não adiciona	[2]
4	True	adiciona 4	[2, 4]
5	False	não adiciona	[2, 4]
6	True	adiciona 6	[2, 4, 6]

Uma compreensão é útil para transformações e filtros simples. Para lógica extensa, um ciclo explícito facilita a leitura.

Guardar e recuperar listas

Gravar (escrever para ficheiro)

```
notas = [12.0, 15.5, 18.0]
with open('notas.txt', 'w', encoding='utf-8') as f:
    for nota in notas:
        f.write(str(nota) + '\n')
```

'w' substitui o conteúdo existente.
Cada nota é escrita numa linha.

Gravar

notas.txt

```
12.0
15.5
18.0
```

Ler

Ler (ler do ficheiro)

```
with open('notas.txt', encoding='utf-8') as f:
    notas = [float(linha.strip()) for linha in f]
```

strip() remove espaços e quebras de linha.
float() converte o texto numa nota (número real).

Conteúdo do ficheiro
após gravação.

Durante a gravação

Iteração	nota	Texto escrito no ficheiro
1	12.0	12.0\n
2	15.5	15.5\n
3	18.0	18.0\n

Cada elemento da
lista é convertido
em texto e escrito
numa linha do
ficheiro.

Após a leitura

Linha lida	linha.strip()	float(...)	Lista notas
'12.0\n'	'12.0'	12.0	[12.0]
'15.5\n'	'15.5'	15.5	[12.0, 15.5]
'18.0\n'	'18.0'	18.0	[12.0, 15.5, 18.0]



Possíveis erros

OSError: problema no ficheiro
(por exemplo, não existe ou sem permissões).

ValueError: texto não numérico
(por exemplo, uma linha que não pode ser convertida).

LISTAS DE LISTAS

Uma lista de listas pode ser vista como uma matriz, com linhas e colunas, onde cada elemento é acessado através de dois índices: o da linha e o da coluna.

```
matriz = [[1, 2, 3], [4, 5, 6]]
```

Aceder a uma linha

```
matriz[1]
```



Resultado:

```
[4, 5, 6]
```

Aceder a um elemento

```
matriz[1][2]
```



Resultado:

```
6
```

		Colunas		
		0	1	2
Linhas	0	1	2	3
	1	4	5	6

Primeiro escolhe-se a linha; depois, a coluna.

Percorrer todos os elementos

```
for linha in matriz:  
    for valor in linha:  
        print(valor, end=' ')  
    print()
```

Saída:

```
1 2 3  
4 5 6
```


Criar e usar tuplos

Um tuplo é uma sequência ordenada e imutável.

Exemplos de criação

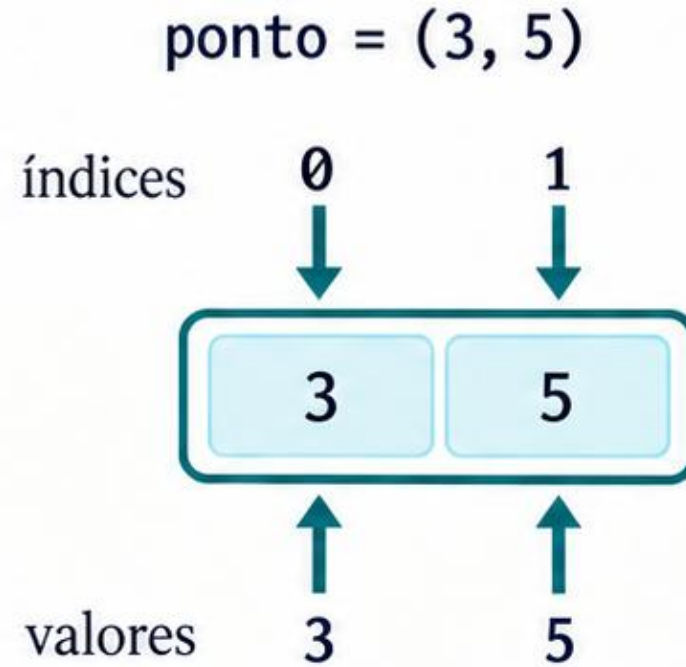
```
ponto = (3, 5)
nomes = ('Ana', 'Rui')
vazio = ()
```

Tuplo com um elemento

A vírgula distingue o tuplo com um elemento.

```
unitario = (10,) → # cria um tuplo
numero = (10) → # cria um inteiro
```

Estrutura de um tuplo



Aceder a elementos

ponto[0] → 3

len(ponto) → 2

Tabela de rastreio

Expressão	Resultado
ponto[0]	3
len(ponto)	2

Utilização

Tuplos são úteis para coordenadas, registos e valores que devem manter a estrutura.

Tuplos: operações e limites

Tuplo: é uma coleção ordenada de elementos, imutável e que pode conter valores de diferentes tipos.

```
dados = (10, 20, 10, 30)
```

0 1 2 3

10 **20** **10** **30**

tuplo dados

OPERAÇÃO	RESULTADO
<code>dados[-1]</code>	30
<code>dados[1:3]</code>	(20, 10)
<code>20 in dados</code>	True
<code>dados.count(10)</code>	2
<code>dados.index(10)</code>	0
<code>sum(dados)</code>	70

Atribuição proibida

```
dados[0] = 99
```



TypeError

```
'tuple' object does not  
support item assignment'
```

Tuplos não têm
append, remove
ou sort.

```
append()  
remove()  
sort()
```



Nota: A imutabilidade impede substituir os elementos do tuplo.
Um objeto mutável contido nele pode mudar.

Lista ou tuplo?

Característica	Lista	Tuplo
Sintaxe	[10, 20]	(10, 20)
Elementos alteráveis	Sim	Não
Tamanho alterável	Sim	Não
Exemplo de uso	Notas a atualizar	Coordenadas fixas

Conversão entre tuplo e lista

```
origem = (10, 20)
lista = list(origem)
lista.append(30)
novo = tuple(lista)
```



Tuplo original
permanece inalterado

```
origem = (10, 20)
```



Novo tuplo com
o elemento adicionado

```
novo = (10, 20, 30)
```



As conversões criam novas sequências.

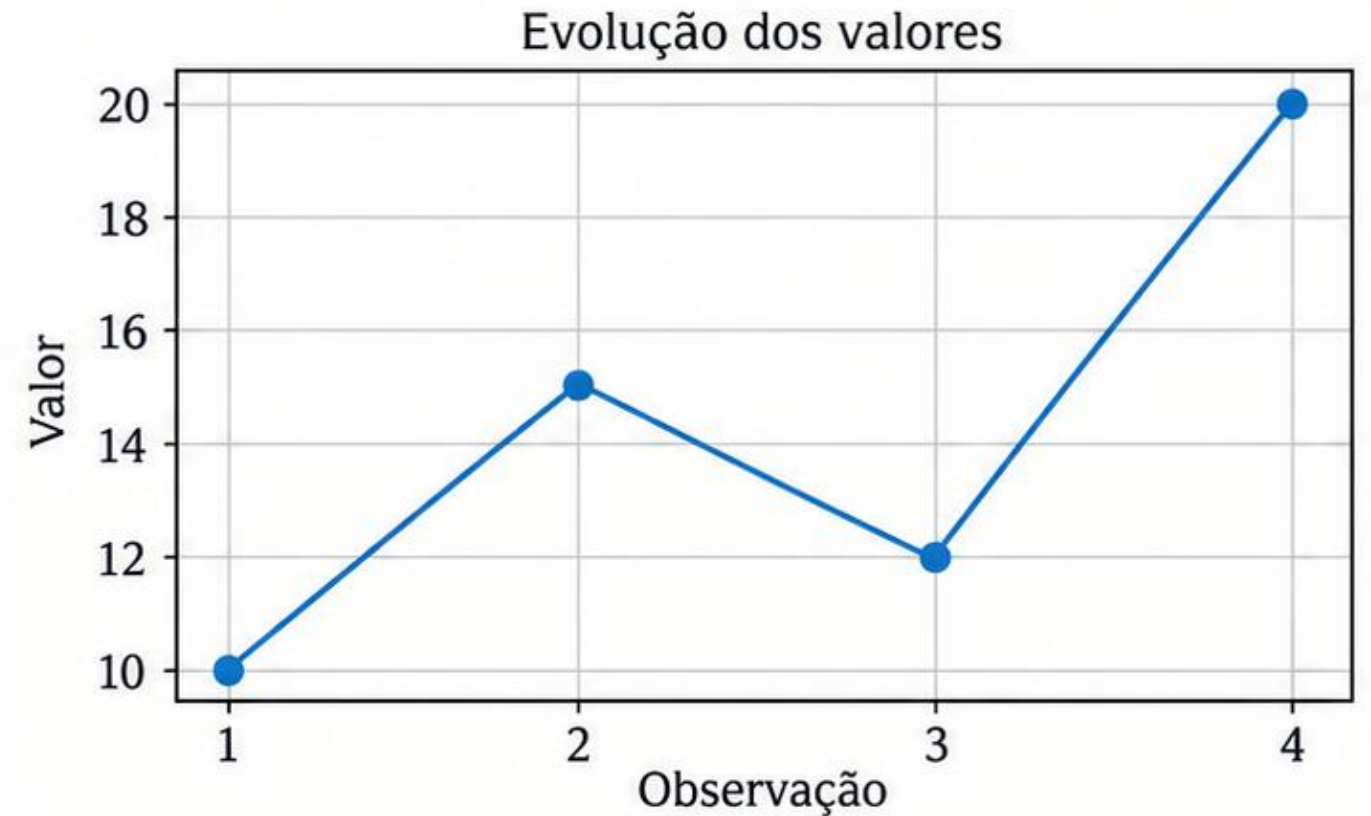
Uma função pode aceitar listas e tuplos se usar operações compatíveis, como `len` e `sum`.

Um gráfico com Matplotlib

Código em Python:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [10, 15, 12, 20]
plt.plot(x, y, marker='o')
plt.title('Evolução dos valores')
plt.xlabel('Observação')
plt.ylabel('Valor')
plt.grid(True)
plt.show()
```

Resultado do gráfico:



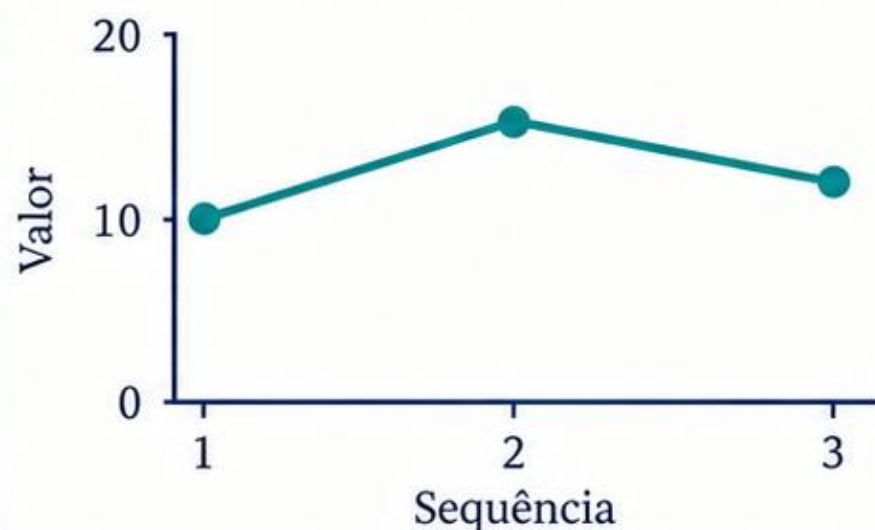
x e y devem ter o mesmo número de elementos.

Cada posição define um ponto ($x[i]$, $y[i]$).

i	$x[i]$	$y[i]$	Ponto ($x[i]$, $y[i]$)
0	1	10	(1, 10)
1	2	15	(2, 15)
2	3	12	(3, 12)
3	4	20	(4, 20)

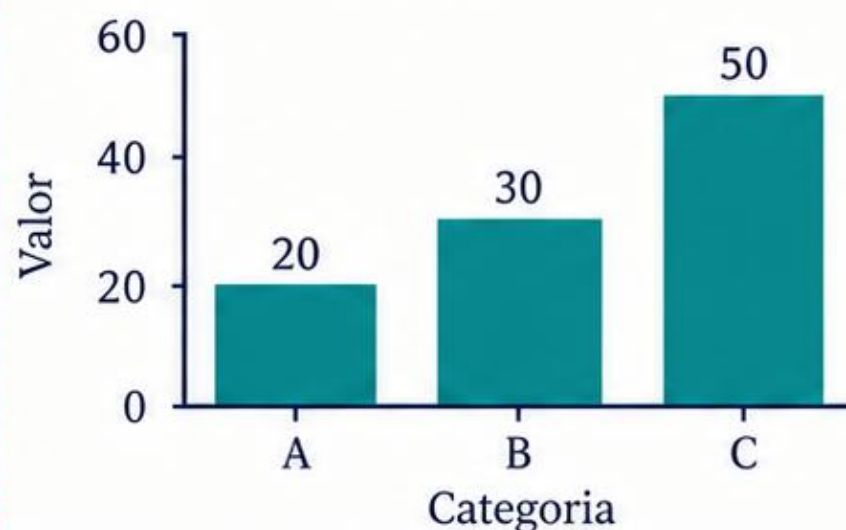
Escolher o tipo de gráfico

Linhas: evolução ao longo de uma sequência.



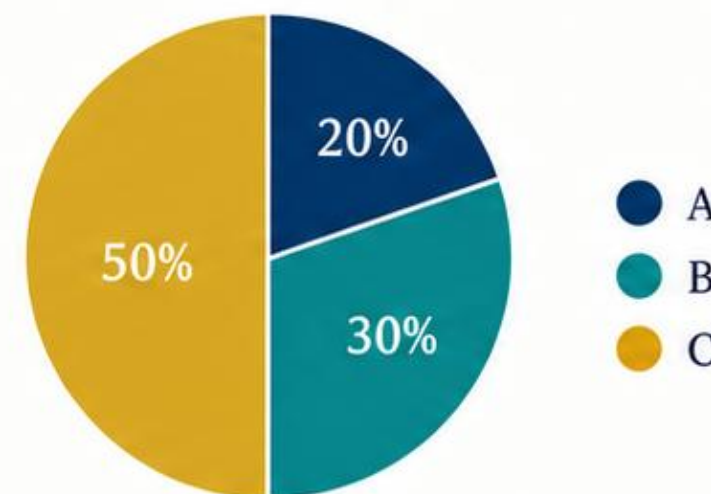
```
import matplotlib.pyplot as plt
plt.plot([1,2,3], [10,15,12])
plt.show()
```

Barras: comparação de categorias.



```
import matplotlib.pyplot as plt
plt.bar(['A','B','C'], [20,30,50])
plt.show()
```

Circular: partes de um total.



```
import matplotlib.pyplot as plt
plt.pie([20,30,50], labels=['A','B','C'],
        autopct='%1.0f%%')
plt.show()
```

O comando `plt.show()` apresenta o gráfico.

Exercício: analisar cinco notas

Objectivo: Ler cinco notas válidas entre 0 e 20 e apresentar uma análise.

Requisitos:

- 1 Guardar as cinco notas numa lista.
- 2 Calcular a média, o mínimo e o máximo.
- 3 Construir a lista de aprovados (nota ≥ 10).
- 4 Guardar um resumo num tuplo (média, mínimo, máximo).
- 5 Criar um gráfico de barras com as notas.

Reutilizar a leitura validada apresentada nesta aula.

Dados de teste e resultados esperados

Lista de notas (dados de teste)

```
notas = [8, 12, 15, 7, 18]
```

Resultados esperados

Elemento	Valor
Média	12.0
Mínimo	7
Máximo	18
Aprovados	[12, 15, 18]
Resumo (tuplo)	(12.0, 7, 18)

Gráfico de barras (dados de teste)



UMA SOLUÇÃO POSSÍVEL

Já existe uma lista `notas` com cinco valores válidos.

Código em Python

```
import matplotlib.pyplot as plt

media = sum(notas) / len(notas)
resumo = (media, min(notas), max(notas))
aprovados = [n for n in notas if n >= 10]
print('Resumo:', resumo)
print('Aprovados:', aprovados)

plt.bar(range(1, len(notas) + 1), notas)
plt.xlabel('Aluno')
plt.ylabel('Nota')
plt.ylim(0, 20)
plt.show()
```

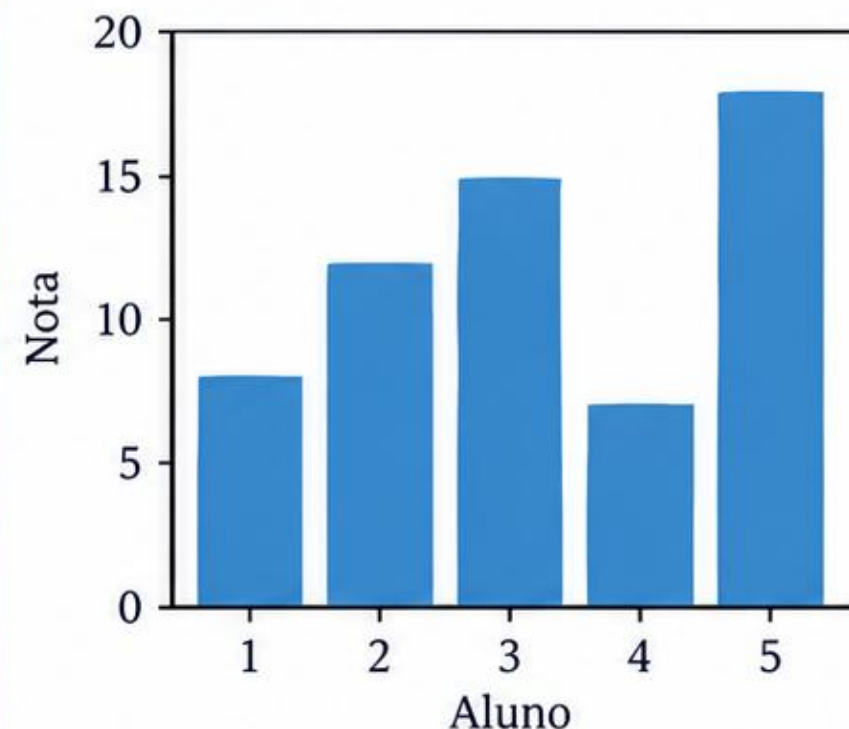
Lista de entrada

Índice	0	1	2	3	4
notas	8	12	15	7	18

Saída obtida

Resumo: (12.0, 7, 18)
Aprovados: [12, 15, 18]

Gráfico de barras



Conclusão:

Lista para guardar e processar;
tuplo para reunir o resumo.